

Python Programming -- Comprehensive Theory Notes

Source: *Core Python Programming* -- Dr. R. Nageswara Rao

Syllabus Coverage: Units 1 - 4 (GCE, ET Dept.)

UNIT 1 -- Introduction to Computers & Python Overview

> *(Unit 1 in the syllabus covers computers, OS, and problem-solving. The Python textbook begins from Chapter 1. The relevant Python content for Unit 1 is covered below as an introduction before diving into Unit 2 topics.)*

UNIT 2 -- Introduction to Python, Data Types & Operators

1. Introduction to Python

What is Python?

Python is a programming language that combines the features of C and Java. It offers the elegant procedural style of C and full object-orientation like Java. Python was developed by **Guido Van Rossum** in **1991** at the Center for Mathematics and Computer Science, Netherlands. It is open-source software downloadable from `www.python.org`.

```
# Python program to add two numbers
a = b = 10
print("Sum= ", (a+b))    # Output: Sum= 20
```

2. Features of Python

| Feature | Description |

|---|---|

| **Simple** | Reads like English; easy to understand |

| **Easy to learn** | Very few keywords; resembles C |

| **Open source** | Free to download and modify |

| **High level language** | Uses English words, not machine code |

| **Dynamically typed** | No variable declaration needed; type determined at runtime |

| **Platform independent** | Byte code runs on any OS via PVM |

| **Portable** | Same result on any computer |

| **Procedure & OOP** | Supports both procedural and object-oriented styles |

| **Interpreted** | Compiler -> byte code -> PVM (interpreter) -> machine code |

| **Extensible** | C/C++ code can be integrated (CPython) |

| **Embeddable** | Python programs can be inserted into C/C++ programs |

| **Huge library** | Built-in "batteries" (numpy, pandas, matplotlib, etc.) |

| **Scripting language** | Interpreted, used to support other software |

| **Database connectivity** | Interfaces with Oracle, MySQL, Sybase |

| **Scalable** | Moves to new environments and uses their features |

3. Execution of a Python Program

Steps:

1. Write source code -> saved as `filename.py`
2. Python compiler converts source code -> **byte code** (`.pyc` file)
3. **PVM (Python Virtual Machine)** contains an interpreter that converts byte code -> **machine code**
4. Processor executes machine code -> output displayed

Source Code (.py) -> Python Compiler -> Byte Code (.pyc) -> PVM (Interpreter) -> Machine

Key commands:

```
python x.py                # compile + run directly
python -m py_compile x.py  # generate .pyc file in __pycache__
python -m dis first.py     # view byte code (disassembler)
```

Byte code: A fixed set of instructions created by Python developers. Each instruction occupies 1 byte. Platform-independent.

JIT Compiler: **Just In Time** compiler -- available in PyPy flavor; converts byte code to machine code faster than the interpreter, improving execution speed.

4. Flavors of Python

| Flavor | Description |

|---|---|

| **CPython** | Standard; written in C; runs on interpreter inside PVM |

| **Jython** | Compiles Python to Java byte code; runs on JVM |

| **IronPython** | For .NET framework; written in C#; runs on CLR |

| **PyPy** | Written in RPython; includes JIT compiler -> faster execution |

| **RubyPython** | Bridge between Ruby and Python interpreters |

| **StacklessPython** | Supports tasklets and channels for concurrent tasks |

| **AnacondaPython** | For large-scale data processing and scientific computing |

5. Python Virtual Machine (PVM)

The **PVM** converts byte code into machine code using an **interpreter** inside it. It identifies the processor and OS of the computer, then generates appropriate machine code.

- PVM is often called the **Python interpreter**

- Standard PVM (CPython) contains only an interpreter -> slower

- PyPy PVM adds a **JIT compiler** -> faster

Memory management:

- Everything is an object in Python (strings, lists, functions, modules)

- **Memory manager** inside PVM allocates heap memory for objects

- OS allocates actual RAM; a **raw memory allocator** sits on top; then **object-specific allocators** manage different types

- Programmers do not manually allocate/free memory (no `malloc()` or `free()`)

6. Frozen Binaries

When distributing software, two options exist:

1. Provide `.pyc` files + ask user to install PVM
2. Bundle `.pyc` files + PVM + libraries into a single executable -> called **“frozen binaries”** (`.exe` for Windows)

Tools: `py2exe` (Windows), `pyinstaller` (UNIX/Linux), `Freeze` (UNIX)

7. Garbage Collection in Python

“Garbage collector (gc module):” Deletes unused objects from memory automatically.

- Maintains a **“reference count”** for each object
- When reference count = 0 -> object is removed from memory
- Handles **“reference cycles”**: A -> B -> C -> A (each has ref count 1 but are unused). GC uses an algorithm to detect and break cycles
- Objects classified into **“3 generations”** (0 = newest, 2 = oldest); GC targets younger objects first
- GC runs automatically based on a **“threshold”** (allocations minus deallocations)
- Can be triggered manually using `gc.collect()`
- Manual GC: **“time-based”** (at intervals) or **“event-based”** (on events like user disconnect)

```
import gc
print(gc.get_threshold())    # view threshold values
gc.collect()                # manually run garbage collector
---
```

8. Comparisons: C vs Python

| C | Python |

|---|---|

| Procedure-oriented | Object-oriented |

| Faster execution | Slower (PyPy is faster but still slower than C) |

| Requires type declaration | No type declaration needed |

| Static, weak typing | Dynamic, strong typing |

| Has pointers | No pointers |

| No exception handling | Robust exception handling |

| Has `do...while`, `while`, `for` | Has `while` and `for` only |

| Has `switch` statement | No `switch` statement |

| `malloc()`, `free()` for memory | Memory managed automatically by PVM |

| No garbage collector | Automatic garbage collector |

| Supports multi-dimensional arrays | Only 1D arrays (use numpy for multi-D) |

| Array index must be positive | Index can be positive or negative |

| No automatic bounds checking | Python checks array bounds |

| Semicolon terminates statements | Newline terminates; semicolon separates expressions |

| Supports inline assignments | No inline assignments |

9. Comments in Python

****Single line comments:**** Start with `#`

```
# This is a single line comment
a = 10 # inline comment
```

****Multi-line comments:**** Use `"""` (triple double quotes) or `'''` (triple single quotes)

```
"""
This is a multi-line comment block.
It can span several lines.
"""
```

> ****Note:**** Triple quotes are not true comments -- they are ****string literals**** that are garbage-collected if not assigned. They are only called ***comments*** by convention.

10. Docstrings

A ****docstring**** is a string written as the ****first statement**** in a module, function, class, or method using `"""` or `'''`. Used to auto-generate API documentation.

```
def add(x, y):
    """
    This function takes two numbers and finds their sum.
    It displays the sum as result.
    """
    print("Sum= ", (x+y))
```

```
def message():
    """
    This function displays a welcome message.
    """
    print("Welcome to Python")
```

```
add(10, 25)
```

```
message()
```

Generate API documentation:

```
python -m pydoc -w ex # creates ex.html with function descriptions
```

11. How Python Sees Variables

In other languages, a variable is a memory "box" that stores a value. In Python, a ****value is an object**** and a ****variable is a tag (name)**** bound to that object.

```
a = 1 # object '1' created; name 'a' bound to it
a = 2 # new object '2' created; 'a' tag moved; old object '1' collected by GC
b = a # both 'b' and 'a' now point to the same object '2'
```

This makes Python memory-efficient: two names can reference the same object.

12. Data Types in Python

A ****datatype**** represents the type of data stored in a variable. Built-in datatypes are of 5 categories:

None Type

`None` represents an object with no value (like `null` in Java). Only one `None` object exists in a program. Represents `False` in Boolean expressions.

Numeric Types

****int:**** Integer number, no decimal point. No size limit.

```

a = -57          # int type
**float:** Floating point number (has decimal point). Supports scientific notation.
num = 55.67998
x = 22.55e3      # = 22550.0
**complex:** Written as `a + bj` or `a + bJ`. `j` = (-1).
c1 = -1-5.5J
c2 = 2.5+2.5J
**Number system literals:**
a = 0b110110    # binary (prefix 0b or 0B)
b = 00145       # octal (prefix 0o or 0O)
c = 0xA180      # hexadecimal (prefix 0x or 0X)
**Type conversion:**
int(15.56)      # -> 15
float(15)       # -> 15.0
complex(10)     # -> (10+0j)
complex(10, -5) # -> (10-5j)
int("1c2", 16)  # -> 450 (hex string to decimal)
bin(10)         # -> '0b1010'
oct(10)         # -> '0o12'
hex(10)         # -> '0xa'
##### bool Datatype

Only two values: `True` (internally 1) and `False` (internally 0).
a = 10 > 5      # a = True
print(True + True) # 2
print(True - False) # 1
##### Sequences

Six types: `str`, `bytes`, `bytearray`, `list`, `tuple`, `range`
**str:** Group of characters in single/double/triple quotes.
s = 'Welcome to Core Python'
print(s[0])     # 'W'
print(s[3:7])   # 'come'
print(s[-1])    # 'n'
print(s*2)      # repeats string twice
**bytes:** Array of byte numbers (0 - 255). Immutable.
elements = [10, 20, 0, 40, 15]
x = bytes(elements) # convert list to bytes
print(x[0])        # 10
# x[0] = 55 -> ERROR (immutable)
**bytearray:** Like bytes but mutable.
x = bytearray(elements)
x[0] = 88         # allowed
**list:** Ordered, mutable, allows mixed types. Written with `[]`.
lst = [10, -20, 15.5, 'Vijay', "Mary"]
**tuple:** Like list but immutable. Written with `()`.
tpl = (10, -20, 15.5, 'Vijay')
# tpl[0] = 99 -> ERROR
**range:** Represents a sequence of numbers. Used with `for` loops.
r = range(10)      # 0 to 9
r = range(30, 40, 2) # 30, 32, 34, 36, 38
lst = list(range(10)) # [0, 1, 2, ..., 9]
##### Sets

Unordered collection; no duplicates.

```

****set:** Mutable.**

```
s = {10, 20, 30, 20, 50}    # {50, 10, 20, 30} (no order, no duplicates)
ch = set("Hello")           # {'H', 'e', 'l', 'o'}
s.update([50, 60])          # add elements
s.remove(50)                # remove element
# No indexing or slicing on sets
```

****frozenset:** Immutable set.**

```
fs = frozenset({50, 60, 70})
# fs.update([80]) -> ERROR
```

Mapping Types (dict)

Key-value pairs, enclosed in `{}`, separated by `:`.

```
d = {10: 'Kamal', 11: 'Pranav', 12: 'Hasini'}
print(d[10])                # 'Kamal'
print(d.keys())              # dict_keys([10, 11, 12])
print(d.values())            # dict_values(['Kamal', 'Pranav', 'Hasini'])
d[10] = 'Ravi'               # update value
del d[11]                    # delete key-value pair
---
```

13. Literals in Python

A ****literal**** is a constant value stored in a variable.

| Literal Type | Examples |

|---|---|

| Integer literal | `450`, `-15` |

| Float literal | `3.14286`, `-10.6`, `1.25E4` |

| Hexadecimal literal | `0x5A1C` |

| Octal literal | `0557` |

| Binary literal | `0B110101` |

| Complex literal | `18+3j` |

| Boolean literal | `True`, `False` |

| String literal | `text`, `text`, `text`, `text` |

****Escape characters:****

| Escape | Meaning |

|---|---|

| `n` | New line |

| `t` | Horizontal tab |

| `\\` | Single backslash |

| `\'` | Single quote |

| `\"` | Double quote |

| `b` | Backspace |

| `r` | Enter |

| `v` | Vertical tab |

| `\\` | Line continuation |

14. User-defined Datatypes

Arrays, classes, and modules created by programmers are user-defined datatypes.

15. Constants in Python

No true constant mechanism in Python. By convention, constants are written in ****ALL CAPS****:

```
MAX_VALUE = 100    # treated as constant by convention, but CAN be changed
---
```

16. Identifiers and Reserved Words

****Identifier****: Name given to a variable, function, class, etc.

- May contain letters, digits, underscore (`\_`)
- Must start with a non-numeric character
- Special symbols `?`, `#`, `\$`, `%`, `@` are NOT allowed
- ****Case sensitive****: `num` ≠ `Num`

****Reserved words (keywords)****:

```
and      del      from      nonlocal  try
as       elif     global    not        while
assert   else     if        or         with
break    except  import    pass       yield
class    exec     in        print      False
continue finally is      raise      True
def      for      lambda    return
---
```

17. Naming Conventions

| Entity | Convention |

|---|---|

| Packages | All lowercase; words separated by `\_` |

| Modules | All lowercase; words separated by `\_` |

| Classes | Each word starts with Capital (e.g. `MyClass`) |

| Global/Module variables | All lowercase with `\_` separator |

| Instance variables | All lowercase; non-public starts with `\_` |

| Functions | All lowercase with `\_` separator |

| Methods | All lowercase with `\_` separator |

| Method arguments | Instance methods: first arg = `self`; class methods: first arg = `cls` |

| Constants | ALL CAPS with `\_` separator (e.g. `MAX\_VALUE`) |

| Non-accessible entities | Double underscore before and after: `\_\_init\_\_` |

18. Determining Datatype

```
a = 15
print(type(a))      # <class 'int'>
ch = 'A'
print(type(ch))      # <class 'str'>  (no char type in Python)
---
```

19. Operators in Python

An ****operator**** is a symbol that performs an operation on ****operands****.

- **Unary operator**: acts on one operand (`-n`)
- **Binary operator**: acts on two operands (`a + b`)
- **Ternary operator**: acts on three operands

Arithmetic Operators (a=13, b=5)

Operator	Meaning	Example	Result
+	Addition	a+b	18
-	Subtraction	a-b	8
*	Multiplication	a*b	65
/	Division	a/b	2.6
%	Modulus (remainder)	a%b	3
**	Exponentiation	a**b	371293
//	Floor (integer) division	a//b	2

---|---|---|---

| `+` | Addition | `a+b` | 18 |

| `-` | Subtraction | `a-b` | 8 |

| `\*` | Multiplication | `a\*b` | 65 |

| `/` | Division | `a/b` | 2.6 |

| `%` | Modulus (remainder) | `a%b` | 3 |

| `\*\*` | Exponentiation | `a\*\*b` | 371293 |

| `//` | Floor (integer) division | `a//b` | 2 |

Order of evaluation: `()` -> `**` -> `* / // %` -> `+ -` -> Assignment

Assignment Operators (x=20, y=10, z=5)

Operator	Meaning	Result
=	Assign	z = x+y -> z=30
+=	Add & assign	z+=x -> z=25
-=	Subtract & assign	z-=x -> z=-15
=	Multiply & assign	z=x -> z=100
/=	Divide & assign	z/=x -> z=0.25
%=	Modulus & assign	z%=x -> z=5
=	Power & assign	z=y -> z=9765625
//=	Floor div & assign	z//=y -> z=0

---|---|---|---

| `=` | Assign | `z = x+y` -> z=30 |

| `+=` | Add & assign | `z+=x` -> z=25 |

| `-=` | Subtract & assign | `z-=x` -> z=-15 |

| `\*=` | Multiply & assign | `z\*=x` -> z=100 |

| `/=` | Divide & assign | `z/=x` -> z=0.25 |

| `%=` | Modulus & assign | `z%=x` -> z=5 |

| `\*\*=` | Power & assign | `z\*\*=y` -> z=9765625 |

| `//=` | Floor div & assign | `z//=y` -> z=0 |

a = b = 1 # assign same value to two variables

a, b = 1, 2 # assign different values

Python has NO ++ or -- operators

Unary Minus Operator

n = 10

print(-n) # -10

num = -10

num = -num

print(num) # 10

Relational Operators (a=1, b=2)

Operator	Meaning	Result
>	Greater than	a>b -> False
>=	Greater than or equal	a>=b -> False
<	Less than	a<b -> True
<=	Less than or equal	a<=b -> True
==	Equal to	a==b -> False
!=	Not equal to	a!=b -> True

---|---|---|---

| `>` | Greater than | `a>b` -> False |

| `>=` | Greater than or equal | `a>=b` -> False |

| `<` | Less than | `a<b` -> True |

| `<=` | Less than or equal | `a<=b` -> True |

| `==` | Equal to | `a==b` -> False |

| `!=` | Not equal to | `a!=b` -> True |

****Chaining relational operators:****

```
x = 15
print(10 < x < 20)    # True (both must be True)
print(1 < 2 < 3 < 4)  # True
print(1 < 2 > 3 < 4)  # False (2>3 is False)
```

Logical Operators (x=1, y=2)

Operator	Meaning	Result
---	---	---
`and`	If x is False, return x; else return y `x and y` -> 2	
`or`	If x is False, return y; else return x `x or y` -> 1	
`not`	If x is False, return True; else True `not x` -> False	

| `and` | If x is False, return x; else return y | `x and y` -> 2 |

| `or` | If x is False, return y; else return x | `x or y` -> 1 |

| `not` | If x is False, return True; else True | `not x` -> False |

```
x=1; y=2; z=3
```

```
if (x<y and y<z): print('Yes')    # compound condition
```

```
if (x>y or y<z): print('Yes')    # or condition
```

Boolean Operators (x=True, y=False)

Operator	Meaning	Result
---	---	---
`and`	True only if both True `x and y` -> False	
`or`	True if either True `x or y` -> True	
`not`	Inverts True/False `not x` -> False	

| `and` | True only if both True | `x and y` -> False |

| `or` | True if either True | `x or y` -> True |

| `not` | Inverts True/False | `not x` -> False |

Bitwise Operators

Operate on individual ****bits**** (binary representation of integers).

Operator	Name	Example (x=10, y=11)
---	---	---
`~`	Complement (NOT) `~x` = -11 (flip all bits)	
`&`	AND `x&y` = 10	
` `	OR `x y` = 11	
`^`	XOR `x^y` = 1	
`<<`	Left shift `x<<2` = 40 (multiply by 4)	
`>>`	Right shift `x>>2` = 2 (divide by 4)	

| `~` | Complement (NOT) | `~x` = -11 (flip all bits) |

| `&` | AND | `x&y` = 10 |

| `|` | OR | `x|y` = 11 |

| `^` | XOR | `x^y` = 1 |

| `<<` | Left shift | `x<<2` = 40 (multiply by 4) |

| `>>` | Right shift | `x>>2` = 2 (divide by 4) |

```
x = 10    # = 0000 1010 in binary
```

```
~x        # = -11 (1111 0101 = -11)
```

```
x << 2    # = 40 (0010 1000)
```

```
x >> 2    # = 2 (0000 0010)
```

Membership Operators

Test whether an element exists in a sequence (string, list, tuple, dict).

Operator	Meaning
---	---
`in`	Returns True if element is found
`not in`	Returns True if element is NOT found

| `in` | Returns True if element is found |

| `not in` | Returns True if element is NOT found |

```
names = ["Rani", "Yamini", "Sushmita"]
```

```
for name in names:
```

```
    print(name)
```

```
if "Core" in "This is Core Python":
```

```
    print("Found")
```

Identity Operators

Compare **memory locations** (identity numbers) of objects using `id()`.

| Operator | Meaning |

---|---

| ``is`` | True if both refer to same object in memory |

| ``is not`` | True if they refer to different objects |

```
a = 25; b = 25
print(a is b)          # True (same object, same id)
```

```
one = [1,2,3,4]
two = [1,2,3,4]
print(one is two)      # False (different list objects)
print(one == two)      # True (same values)
print(id(a))           # identity/memory address number
```

Operator Precedence (Highest to Lowest)

| Operator | Name |

---|---

| ``(`` | Parenthesis |

| ``**`` | Exponentiation |

| ``-`, ~`` | Unary minus, Bitwise complement |

| ``*`, /, //, %`` | Multiply, Divide, Floor, Modulus |

| ``+`, -`` | Addition, Subtraction |

| ``<<`, >>`` | Bitwise shifts |

| ``&`` | Bitwise AND |

| ``^`` | Bitwise XOR |

| ``\|`` | Bitwise OR |

| ``>`, >=, <, <=, ==, !=`` | Relational |

| ``=`, %=, /=, //=, -=, +=, *=, **=`` | Assignment |

| ``is`, is not`` | Identity |

| ``in`, not in`` | Membership |

| ``not`` | Logical NOT |

| ``or`` | Logical OR |

| ``and`` | Logical AND |

Associativity: Almost all operators are **left-to-right** when same precedence.

20. Mathematical Functions (math module)

```
import math
import math as m      # alias
from math import sqrt, factorial  # specific import
```

| Function | Description | Example |

---|---

| ``ceil(x)`` | Round up to next integer | ``ceil(4.5)`` -> 5 |

| ``floor(x)`` | Round down | ``floor(4.5)`` -> 4 |

```
| `sqrt(x)` | Square root | `sqrt(49)` -> 7.0 |
| `pow(x,y)` | x to the power y | `pow(5,3)` -> 125.0 |
| `factorial(x)` | Factorial | `factorial(4)` -> 24 |
| `sin(x)` | Sine (radians) | `sin(0.5)` -> 0.479 |
| `cos(x)` | Cosine (radians) | `cos(0.5)` -> 0.877 |
| `tan(x)` | Tangent (radians) | `tan(0.5)` -> 0.546 |
| `degrees(x)` | Radians to degrees | `degrees(3.14159)` -> 179.99 |
| `radians(x)` | Degrees to radians | `radians(180)` -> 3.14159 |
| `log(x, base)` | Logarithm | `log(5.5, 2)` -> 2.459 |
| `log10(x)` | Base-10 log | `log10(5.2345)` -> 0.718 |
| `exp(x)` | e^x | `exp(0.5)` -> 1.648 |
| `fabs(x)` | Absolute value | `fabs(-4.56)` -> 4.56 |
| `trunc(x)` | Truncate to int | `trunc(15.56)` -> 15 |
| `gcd(x, y)` | Greatest common divisor | `gcd(25, 30)` -> 5 |
**Math constants:** `math.pi` = 3.14159..., `math.e` = 2.71828..., `math.inf`, `math.nan`
---
```

21. Input and Output

```
##### Output -- print()

print()                # blank line
print("Hello")         # print string
print("Hello", end='') # no newline at end
print(a, b, sep=',')   # custom separator
print('x= %i y= %d' % (x, y)) # % formatting
print('Hello {0}, {1}'.format(name, salary)) # format() method
print(f'Hello {name}')    # f-string (Python 3.6+)
print('%.2f' % num)       # float with 2 decimal places
##### Input -- input()

`input()` always returns a **string**. Convert using `int()` or `float()`.

str = input('Enter a string: ') # accept string
x = int(input('Enter a number: ')) # accept integer
x = float(input('Enter a number: ')) # accept float
ch = input("Enter a char: ")[0] # first character only

# Accept multiple values in one line
a, b = [int(x) for x in input("Enter two numbers: ").split(',')]
---
```

22. Control Statements

Control statements change the flow of execution.

> **Note:** Python does NOT have a `switch` statement.

if Statement

```
if condition:
    statements

# Example
num = 1
if num == 1:
    print("One")
```

Indentation

Statements with same indentation belong to the same ****suite**** (group). Default = 4 spaces.

```
if x == 1:
    print('a')      # 4 spaces  --  inside if
    print('b')      # 4 spaces  --  inside if
print('end')        # 0 spaces  --  outside if
```

if...else Statement

```
if condition:
    statements1
else:
    statements2
```

Example

```
x = int(input("Enter a number: "))
if x % 2 == 0:
    print(x, "is even")
else:
    print(x, "is odd")
```

if...elif...else Statement

```
if condition1:
    statements1
elif condition2:
    statements2
elif condition3:
    statements3
else:
    statements4
```

Example

```
num = -5
if num == 0:
    print(num, "is zero")
elif num > 0:
    print(num, "is positive")
else:
    print(num, "is negative")
```

while Loop

Executes as long as the condition is True.

```
while condition:
    statements
```

Example: display 1 to 10

```
x = 1
while x <= 10:
    print(x)
    x += 1
```

Example: even numbers between m and n

```
m, n = [int(i) for i in input("Enter min, max: ").split(',')]
x = m
if x % 2 != 0: x = x + 1
while x >= m and x <= n:
    print(x)
    x += 2
```

for Loop

Iterates over elements of a sequence (string, list, tuple, range, etc.).

```
for var in sequence:
    statements
```

```
# Example: display characters of string
str = 'Hello'
for ch in str:
    print(ch)
```

```
# Example: using range()
for i in range(1, 11):      # 1 to 10
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8
    print(i)
```

```
# Example: display list elements
lst = [10, 20.5, 'A', 'America']
for element in lst:
    print(element)
```

Infinite Loops

```
while True:
    print("Hai")      # runs forever -- avoid in programs
Press `Ctrl+C` to stop an infinite loop at the system prompt.
```

Nested Loops

```
for i in range(3):      # outer: 0, 1, 2
    for j in range(4):  # inner: 0, 1, 2, 3
        print('i=', i, 'j=', j)
```

```
# Star triangle using nested loops
for i in range(1, 11):
    for j in range(1, i+1):
        print('* ', end='')
    print()
```

```
# Elegant single-loop version
for i in range(1, 11):
    print('* '*(i))
```

else Suite with Loops

The `else` block always executes (even if loop body didn't run), UNLESS `break` was used.

```
for element in [1, 2, 3, 4, 5]:
    if search == element:
        print('Found')
        break
else:
    print('Not found')      # runs only if no break occurred
```

break Statement

Exits the current loop immediately.

```
x = 10
while x >= 1:
    print('x=', x)
    x -= 1
    if x == 5:
        break      # exit when x becomes 5
print("Out of loop")
```

continue Statement

Skips the rest of the loop body and goes back to the loop condition.

```
x = 0
while x < 10:
    x += 1
    if x > 5:
        continue    # skip print for x > 5
    print('x=', x)
print("Out of loop")
##### pass Statement

Does nothing. Used as a placeholder.

num = [1, 2, 3, -4, -5, -6]
for i in num:
    if i > 0:
        pass        # not interested in positive numbers
    else:
        print(i)    # display only negatives
##### assert Statement

Tests a condition; raises `AssertionError` if False.

x = int(input('Enter a number greater than 0: '))
assert x > 0, "Wrong input entered"
print('You entered:', x)

# With exception handling
try:
    assert(x > 0)
    print('You entered:', x)
except AssertionError:
    print("Wrong input entered")
##### return Statement

Returns a value from a function (discussed more in Unit 4).

def sum(a, b):
    return a + b

res = sum(5, 10)
print('The result is', res)    # 15
---
```

UNIT 3 -- Arrays in Python & Strings and Characters

23. Arrays in Python

****Advantages of arrays:****

- Store one type of elements efficiently
- Use less memory than lists for large datasets
- Faster execution than lists
- Grow or shrink dynamically in memory
- Useful for handling collections of elements

Creating a 1D Array (array module)

```
from array import *

a = array('i', [5, 6, -7, 8])    # integer array
arr = array('d', [1.5, -2.2, 3, 5.75]) # double/float array
arr2 = array('u', ['a', 'b', 'c']) # unicode character array
**Type codes:**
```

Code	C Type	Size (bytes)
`b`	signed integer	1
`B`	unsigned integer	1
`i`	signed integer	2
`I`	unsigned integer	2
`l`	signed integer	4
`L`	unsigned integer	4
`f`	floating point	4
`d`	double precision float	8
`u`	unicode character	2

****Three ways to import:****

```
import array                                # use as array.array('i', [...])
import array as ar                          # use as ar.array('i', [...])
from array import *                         # use directly as array('i', [...])
```

Indexing and Slicing Arrays

```
x = array('i', [10, 20, 30, 40, 50, 60, 70])
n = len(x)                                # 7
```

```
# Indexing
print(x[0])                               # 10
print(x[-1])                              # 70 (last element)
```

```
# Slicing: arrayname[start:stop:stride]
y = x[1:4]                                # [20, 30, 40]
y = x[0:]                                  # [10, 20, 30, 40, 50, 60, 70]
y = x[:4]                                  # [10, 20, 30, 40]
y = x[-4:]                                 # [40, 50, 60, 70] (last 4)
y = x[-4:-1]                              # [40, 50, 60]
y = x[0:7:2]                              # [10, 30, 50, 70] (every other)
```

```
# Update via slicing
x[1:3] = array('i', [5, 7])               # replace 1st and 2nd elements
##### Processing Arrays (array module methods)
```

Method	Description
`a.append(x)`	Add element at end
`a.insert(i, x)`	Insert x at position i
`a.remove(x)`	Remove first occurrence of x
`a.pop()`	Remove and return last element
`a.pop(x)`	Remove element x
`a.index(x)`	Return position of first occurrence of x
`a.count(x)`	Count occurrences of x
`a.reverse()`	Reverse the array
`a.extend(x)`	Append another array or iterable
`a.fromlist(lst)`	Append items from list
`a.tolist()`	Convert array to list

| `a.typecode` | Type code character of array |

| `a.itemsize` | Size (bytes) of each element |

```
arr = array('i', [10, 20, 30, 40, 50])
arr.append(60)
arr.insert(1, 99)
arr.remove(20)
n = arr.pop()
print(arr.index(30))
arr.reverse()
```

Types of Arrays

- **Single dimensional (1D):** One row/column of elements. Python's `array` module supports only this.

- **Multi-dimensional (2D, 3D):** Multiple rows and columns. Requires **numpy** in Python.

24. Working with Arrays using numpy

numpy is a package for scientific calculations -- handles 1D and multi-dimensional arrays.

```
import numpy          # use as numpy.array(...)
import numpy as np    # use as np.array(...)
from numpy import *   # use directly as array(...)

##### Creating Arrays in numpy

Using array():
from numpy import *
arr = array([10, 20, 30, 40, 50], int)    # integer array
arr = array([1.5, 2.5, 3, -5.1], float)   # float array
arr = array(['a', 'b', 'c', 'd'])         # character array
arr = array(['Delhi', 'Mumbai'], dtype=str) # string array
```

Using linspace(): Evenly spaced points between start and stop.

```
a = linspace(0, 10, 5)    # [0., 2.5, 5., 7.5, 10.]
```

Using logspace(): Evenly spaced on logarithmic scale.

```
a = logspace(1, 4, 5)     # 5 points from 10^1 to 10^4
```

Using arange(): Like `range()` but for arrays.

```
arange(10)                # [0 1 2 3 4 5 6 7 8 9]
arange(5, 10)             # [5 6 7 8 9]
arange(1, 10, 3)          # [1 4 7]
arange(10, 1, -1)         # [10 9 8 7 6 5 4 3 2]
arange(0, 10, 1.5)        # [0. 1.5 3. 4.5 6. 7.5 9.]
```

Using zeros() and ones():

```
zeros(5, int)             # [0 0 0 0 0]
ones(5, float)            # [1. 1. 1. 1. 1.]
```

Mathematical Operations on Arrays (Vectorized)

```
arr = array([10, 20, 30.5, -40])
print(arr + 5)            # [15. 25. 35.5 -35.]
print(arr - 5)
print(arr * 5)
print(arr / 5)
print(arr % 5)
print((arr + 5)**2 - 10)
```

```
arr1 = array([10, 20, 30.5, -40])
arr2 = array([1, 2, 3, 4])
arr3 = arr1 - arr2        # element-wise subtraction
numpy mathematical functions on arrays:
```



```

| Function | Description |
|---|---|
| `sin(arr)` | Sine of each element |
| `cos(arr)` | Cosine of each element |
| `tan(arr)` | Tangent of each element |
| `sqrt(arr)` | Square root of each element |
| `exp(arr)` | e^x for each element |
| `abs(arr)` | Absolute value of each element |
| `log(arr)` | Natural log of each element |
| `sum(arr)` | Sum of all elements |
| `prod(arr)` | Product of all elements |
| `min(arr)` | Smallest element |
| `max(arr)` | Biggest element |
| `mean(arr)` | Average |
| `median(arr)` | Median |
| `std(arr)` | Standard deviation |
| `var(arr)` | Variance |
| `sort(arr)` | Sorted array |
| `unique(arr)` | Unique elements |
| `concatenate([a,b])` | Join arrays |
| `argmin(arr)` | Index of smallest |
| `argmax(arr)` | Index of biggest |

from numpy import *
arr = array([10, 20, 30.5, -40])
print(sin(arr))      # sine of each element
print(max(arr))      # 30.5
print(sum(arr))      # 20.5
print(mean(arr))     # 5.125
##### Multi-dimensional Arrays (numpy)

**reshape():** Convert 1D array to 2D or 3D.

a = array([1, 2, 3, 4, 5, 6])
b = reshape(a, (2, 3))    # 2 rows, 3 cols  -> [[1,2,3],[4,5,6]]
b = reshape(a, (3, 2))    # 3 rows, 2 cols  -> [[1,2],[3,4],[5,6]]

a = arange(12)
b = reshape(a, (2, 3, 2))    # 3D: 2 arrays each 3x2

**Indexing in 2D arrays:**

a = [[1,2,3], [4,5,6], [7,8,9]]
print(a[0])                # [1, 2, 3]  -- first row
print(a[1][2])              # 6  -- row 1, col 2

for i in range(len(a)):
    for j in range(len(a[i])):
        print(a[i][j], end=' ')

**Slicing 2D arrays:**

a = reshape(a, (3,3))
a[:, :]    # all rows, all cols

```

```

a[0, :]      # row 0 only -> [1 2 3]
a[:, 0]      # col 0 only -> [1 4 7]
a[0:1, 0:1] # element at row 0, col 0 -> [[1]]
a[1:2, 1:2] # element at row 1, col 1 -> [[5]]
---
```

25. Strings and Characters

A **string** is a group of characters. Python uses `str` datatype (no separate `char` type).

Creating Strings

```

s1 = 'Welcome to Core Python learning'
s2 = "Welcome to Core Python learning"
str = '''Multi
line string'''
str = r"raw \n string"      # raw string: escape chars ignored
name = u'\u0915\u094b\u0930' # unicode string
```

Length of a String

```

str = 'Core Python'
n = len(str)      # 11
```

Indexing in Strings

```

str = 'Core Python'
print(str[0])      # 'C'
print(str[-1])     # 'n' (last character)
print(str[-4])     # 't'
```

```

# Access forward using while
i = 0
while i < len(str):
    print(str[i], end=' ')
    i += 1
```

```

# Access in reverse
for i in str[::-1]:
    print(i, end=' ')
```

Slicing Strings

Format: `stringname[start:stop:stepsize]`

```

s = 'Core Python'
s[0:9:1]      # 'Core Pyth'
s[0:9:2]      # 'Cr yh'
s[::]         # 'Core Python' (all)
s[2:4:1]      # 're'
s[::2]        # 'Cr yhn'
s[2::]        # 're Python'
s[:4:]        # 'Core'
s[-4:-1]      # 'tho'
s[-1::-1]     # 'nohtyP eroC' (reverse)
```

Repeating Strings

```

str = 'Core Python'
print(str * 2)      # 'Core PythonCore Python'
s = str[5:7] * 3     # repeat 'Py' 3 times -> 'PyPyPy'
```

Concatenation of Strings

```

s1 = 'Core'
s2 = 'Python'
s3 = s1 + s2      # 'CorePython'
```

Checking Membership

```

str = 'This is Core Python'
```

```
sub = 'Core'
if sub in str:
    print("Found")
if sub not in str:
    print("Not found")
#### Comparing Strings
s1 = 'Box'; s2 = 'Boy'
if s1 == s2: print("Same")
if s1 < s2: print("s1 comes before s2 in dictionary order")
#### Removing Spaces
name = ' Mukesh Deshmukh '
print(name.rstrip())    # remove right spaces -> ' Mukesh Deshmukh'
print(name.lstrip())    # remove left spaces -> 'Mukesh Deshmukh '
print(name.strip())     # remove both sides -> 'Mukesh Deshmukh'
#### Finding Substrings
str = 'This is a book'
sub = 'is'

# find() returns -1 if not found; index() raises ValueError
n = str.find(sub, 0, len(str))    # first occurrence from start
n = str.rfind(sub)                # first occurrence from right
n = str.index(sub, 0, len(str))   # raises ValueError if not found

# Count all occurrences
n = str.count('is')               # 2 (found at position 3 and 6)
n = str.count('e', 0, 3)         # in range 0 to 2
#### Strings are Immutable
str = 'abcd'
str[0] = 'x'    # TypeError: 'str' object does not support item assignment
Reassignment creates a new object:
s1 = 'one'; s2 = 'two'
s2 = s1    # s2 now points to 'one'; 'two' collected by GC
#### Replacing a Substring
str = 'That is a beautiful girl'
str1 = str.replace('girl', 'flower')
# str is unchanged; str1 = 'That is a beautiful flower'
#### Splitting and Joining Strings
# split(): break string into list
str = 'one,two,three,four'
lst = str.split(',')    # ['one', 'two', 'three', 'four']

# join(): merge list into string
str = ('one', 'two', 'three')
str1 = "-".join(str)    # 'one-two-three'

lst = ['apple', 'guava', 'mango']
str1 = ':'.join(lst)    # 'apple:guava:mango'
#### Changing Case
str = 'Python is the future'
str.upper()    # 'PYTHON IS THE FUTURE'
str.lower()    # 'python is the future'
str.swapcase() # 'pYTHON IS THE FUTURE'
str.title()    # 'Python Is The Future'
#### Checking Start/End
str = 'This is Python'
str.startswith('This')    # True
```

```
str.endswith('Python')    # True
##### String Testing Methods
| Method | Returns True if... |
|---|---|
| `isalnum()` | All chars are alphanumeric (A-Z, a-z, 0-9) |
| `isalpha()` | All chars are alphabetic |
| `isdigit()` | All chars are digits |
| `islower()` | All letters are lowercase |
| `isupper()` | All letters are uppercase |
| `istitle()` | Each word starts with capital |
| `isspace()` | String contains only spaces |
---
```

UNIT 4 -- Functions, Lists and Tuples

26. Functions

A **function** is a named block of statements that performs a specific task. Functions represent **reusable code**.

Function vs Method:

- Function: written individually, called by name
- Method: written inside a class, called as `object.method()` or `Class.method()`

Defining and Calling a Function

```
def sum(a, b):
    """ This function finds sum of two numbers """
    c = a + b
    print('Sum= ', c)
```

Call the function

```
sum(10, 15)          # Output: Sum= 25
sum(1.5, 10.75)      # Output: Sum= 12.25
```

- `def` keyword starts function definition
- Parameters in `()` receive values from outside
- Body indented with 4 spaces
- Docstring as first statement (optional but recommended)

Returning Results

```
def sum(a, b):
    c = a + b
    return c
```

```
x = sum(10, 15)      # x = 25
```

Returning multiple values:

```
def sum_sub(a, b):
    return a + b, a - b    # returns a tuple
```

```
x, y = sum_sub(10, 5)    # x=15, y=5
```

```
def sum_sub_mul_div(a, b):
    return a+b, a-b, a*b, a/b
```

```
t = sum_sub_mul_div(10, 5)
for i in t:
    print(i, end=', ')    # 15, 5, 50, 2.0,
```

Formal and Actual Arguments

- **Formal arguments:** Parameters in the function definition

- **Actual arguments:** Values passed at the time of calling

```
def sum(a, b):          # a, b = formal arguments
    c = a + b
    print(c)
```

```
x = 10; y = 15
sum(x, y)              # x, y = actual arguments
```

Types of Actual Arguments

1. Positional Arguments: Passed in exact order, exact number.

```
def attach(s1, s2):
    s3 = s1 + s2
    print('Total string: ' + s3)
```

```
attach('New', 'York')    # -> NewYork
```

2. Keyword Arguments: Identified by parameter names; order doesn't matter.

```
def grocery(item, price):
    print('Item = %s' % item)
    print('Price = %.2f' % price)
```

```
grocery(item='Sugar', price=50.75)
grocery(price=88.00, item='Oil')    # order changed -- still works
```

3. Default Arguments: Parameter has a default value used if no argument passed.

```
def grocery(item, price=40.00):
    print('Item = %s' % item)
    print('Price = %.2f' % price)
```

```
grocery(item='Sugar', price=50.75)    # uses 50.75
grocery(item='Sugar')                  # uses default 40.00
```

4. Variable Length Arguments (*args): Accepts any number of values; stored as tuple.

```
def add(farg, *args):
    print('Formal argument=', farg)
    sum = 0
    for i in args:
        sum += i
    print('Sum of all numbers=', (farg + sum))
```

```
add(5, 10)          # formal=5, args=(10,)
add(5, 10, 20, 30)   # formal=5, args=(10,20,30)
```

Keyword variable length arguments (kwargs):** Accepts key=value pairs; stored as dict.

```
def display(farg, **kwargs):
    print('Formal argument=', farg)
    for x, y in kwargs.items():
        print('key = {}, value = {}'.format(x, y))
```

```
display(5, rno=10, name='Prakash')
```

Local and Global Variables

Local variable: Declared inside a function; available only within that function.

```
def myfunction():
    a = 1          # local variable
    a += 1
```

```

    print(a)    # 2

myfunction()
# print(a)    # ERROR: 'a' is not defined outside
**Global variable:** Declared outside all functions; available to all functions below it.
a = 1    # global variable

def myfunction():
    b = 2    # local
    print('a=', a) # can access global
    print('b=', b) # can access local

myfunction()
print(a)    # available
# print(b)  # ERROR
**Using `global` keyword to modify a global variable inside a function:**
a = 1

def myfunction():
    global a    # declare it global
    print('global a=', a)
    a = 2    # modify global variable
    print('modified a=', a)

myfunction()
print('global a=', a)    # 2 (modified)
**Using `globals()` to access global when local has same name:**
a = 1
def myfunction():
    a = 2    # local variable
    x = globals()['a'] # access global 'a'
    print('global a=', x)
    print('local a=', a)
##### Passing a Group of Elements to a Function
def calculate(lst):
    n = len(lst)
    total = sum(i for i in lst)
    avg = total / n
    return total, avg

lst = [int(x) for x in input().split()]
x, y = calculate(lst)
print('Total:', x, 'Average:', y)
---
```

27. Lists

A **list** is like an array but can store **different types of elements**. Lists are **mutable** (can be modified). Written with `[]`.

Creating Lists

```

num = [10, 20, 30, 40, 50]
names = ["Raju", "Vani", "Gopal"]
x = [10, 20, 10.5, 2.55, "Ganesh", 'Vishnu']    # mixed types
e_lst = []    # empty list
student = [10, 'Venu gopal', 'M', 50, 55, 62]    # realistic example
##### Creating Lists using range()
lst = list(range(10))    # [0, 1, 2, ..., 9]
```

```

lst = list(range(4, 9, 2)) # [4, 6, 8]
lst = range(5, 10)        # range object (use in loop or list())
#### Indexing and Slicing Lists
student = [10, 'Venu gopal', 'M', 50, 55, 62, 74, 66]
print(student[0])          # 10
print(student[1])          # 'Venu gopal'
print(student[-1])         # 66 (last element)
print(student[0:3:1])      # [10, 'Venu gopal', 'M']
print(student[:3])         # [10, 'Venu gopal', 'M']
print(student[:])          # entire list
#### Updating Lists (Mutable)
lst = list(range(1, 5))    # [1, 2, 3, 4]

lst.append(9)              # [1, 2, 3, 4, 9]
lst[1] = 8                 # update element -> [1, 8, 3, 4, 9]
lst[1:3] = 10, 11          # update slice -> [1, 10, 11, 4, 9]
del lst[1]                 # delete by index -> [1, 11, 4, 9]
lst.remove(11)             # delete by value -> [1, 4, 9]
#### Concatenation and Repetition
x = [10, 20, 30, 40, 50]
y = [100, 110, 120]
print(x + y)               # [10, 20, 30, 40, 50, 100, 110, 120]
print(x * 2)               # [10, 20, 30, 40, 50, 10, 20, 30, 40, 50]
#### Membership in Lists
x = [10, 20, 30, 40, 50]
a = 20
print(a in x)              # True
print(a not in x)         # False
#### Aliasing and Cloning Lists
x = [10, 20, 30, 40, 50]
y = x                      # aliasing: y is another name for x; changes to x affect y
y = x[:]                   # cloning: independent copy; changes to x don't affect y
y = x.copy()              # same as cloning
#### Methods to Process Lists
| Method | Description |
|---|---|
| `len(lst)` | Number of elements |
| `max(lst)` | Biggest element |
| `min(lst)` | Smallest element |
| `lst.append(x)` | Add element at end |
| `lst.insert(i, x)` | Insert x at position i |
| `lst.copy()` | Return a copy |
| `lst.extend(lst1)` | Append lst1 to lst |
| `lst.count(x)` | Count occurrences of x |
| `lst.remove(x)` | Remove first occurrence of x |
| `lst.pop()` | Remove and return last element |
| `lst.sort()` | Sort ascending |
| `lst.sort(reverse=True)` | Sort descending |
| `lst.reverse()` | Reverse the list |

```

```
| `lst.clear()` | Delete all elements |
| `lst.index(x)` | First occurrence position of x |
num = [10, 20, 30, 40, 50]
num.append(60)
num.insert(0, 5)
num1 = num.copy()
num.extend(num1)
n = num.count(50)
num.remove(50)
num.pop()
num.sort()
num.reverse()
num.clear()
#### Finding Biggest and Smallest Elements
x = [20, 10, 5, 20, 15]
big = x[0]; small = x[0]
for i in range(1, len(x)):
    if x[i] > big: big = x[i]
    if x[i] < small: small = x[i]
print('Max:', big, 'Min:', small)

# Or simply:
print(max(x))    # 20
print(min(x))    # 5
#### Sorting the List -- Bubble Sort
# bubble sort
for i in range(n-1):
    for j in range(n-1-i):
        if x[j] > x[j+1]:
            x[j], x[j+1] = x[j+1], x[j]    # swap
# Or simply:
x.sort()
#### Nested Lists (Lists within Lists)
b = [10, 20, 30, [80, 90]]
print(b[3])      # [80, 90]
print(b[3][0])   # 80
**Using nested lists as matrices:**
mat = [[1,2,3], [4,5,6], [7,8,9]]    # 3x3 matrix
for r in mat:
    for c in r:
        print(c, end=' ')
    print()

# Using indexing
for i in range(len(mat)):
    for j in range(len(mat[i])):
        print('%d ' % mat[i][j], end='')
    print()
**Matrix addition:**
m3 = [4*[0] for i in range(3)]    # 3x4 zero matrix
for i in range(3):
    for j in range(4):
        m3[i][j] = m1[i][j] + m2[i][j]
#### List Comprehensions
Compact way to create lists from iterables.
squares = [x**2 for x in range(1, 11)]
```



```
# [1, 4, 9, 16, 25, 36, 49, 64, 81, 100]

even_squares = [x**2 for x in range(1, 11) if x % 2 == 0]
# [4, 16, 36, 64, 100]

lst = [i+j for i in [10, 20, 30] for j in [1, 2, 3, 4]]
# [11, 12, 13, 14, 21, ...]

lst = [w[0] for w in ['Apple', 'Grapes', 'Banana']]
# ['A', 'G', 'B']

num3 = [i for i in num1 if i not in num2]    # elements in num1 but not num2
---
```

28. Tuples

A **tuple** is like a list but **immutable** (cannot be modified). Written with ``( )``.

Use tuples to store data that should not change.

Creating Tuples

```
tup1 = ()                # empty tuple
tup2 = (10,)             # single element (MUST have comma)
tup3 = (10, 20, -30.1, 40.5, 'Hyderabad') # mixed types
tup4 = (10, 20, 30)      # integers only
tup5 = 1, 2, 3, 4        # no brackets -> still a tuple
```

```
# From a list
list = [1, 2, 3]
tpl = tuple(list)      # (1, 2, 3)
```

```
# Using range()
tpl = tuple(range(4, 9, 2))    # (4, 6, 8)
```

Accessing Tuple Elements

****Indexing:****

```
tup = (50, 60, 70, 80, 90, 100)
print(tup[0])    # 50
print(tup[5])    # 100
print(tup[-1])   # 100 (last)
print(tup[-6])   # 50 (first from end)
```

****Slicing:****

```
print(tup[:])    # (50, 60, 70, 80, 90, 100)
print(tup[1:4])  # (60, 70, 80)
print(tup[:2])   # (50, 70, 90) -- alternate elements
print(tup[::-2]) # (100, 80, 60) -- reverse alternate
print(tup[-4:-1]) # (70, 80, 90)
```

****Extracting into variables:****

```
student = (10, 'Vinay kumar', 50, 60, 65, 61, 70)
rno, name = student[0:2]    # rno=10, name='Vinay kumar'
marks = student[2:7]        # (50, 60, 65, 61, 70)
```

Basic Operations on Tuples

```
student = (10, 'Vinaykumar', 50, 60, 65, 61, 70)
```

```
# Length
len(student)    # 7
```

```
# Concatenation
student1 = student + fees_tuple
```

```
# Repetition
fees = (25000.00,) * 4      # (25000.0, 25000.0, 25000.0, 25000.0)

# Membership
'Vinay kumar' in student1   # True
'Vinay kumar' not in student1 # False

tpl = (10, 11, 12)
tpl1 = tpl * 3              # (10, 11, 12, 10, 11, 12, 10, 11, 12)
#### Functions to Process Tuples
| Function/Method | Description |
|---|---|
| `len(tpl)` | Number of elements |
| `min(tpl)` | Smallest element |
| `max(tpl)` | Biggest element |
| `tpl.count(x)` | Count occurrences of x |
| `tpl.index(x)` | First occurrence position of x (raises `ValueError` if not found) |
| `sorted(tpl)` | Sort ascending (returns list) |
| `sorted(tpl, reverse=True)` | Sort descending |
num = eval(input("Enter elements in (): "))    # accepts tuple from keyboard
sum = 0
n = len(num)
for i in range(n):
    sum += num[i]
print('Sum:', sum, 'Average:', sum/n)
**Finding an element with index():**
str = input('Enter elements separated by commas: ').split(',')
lst = [int(num) for num in str]
tup = tuple(lst)
ele = int(input('Enter element to search: '))
try:
    pos = tup.index(ele)
    print('Found at position:', pos+1)
except ValueError:
    print('Element not found')
---
```

Quick Reference Code Snippets

```
# Fibonacci series
f1 = 0; f2 = 1
n = int(input('How many Fibonacci? '))
print(f1, '\n', f2, sep='')
c = 2
while c < n:
    f = f1 + f2
    print(f)
    f1, f2 = f2, f
    c += 1

# Prime number check
def prime(n):
    x = 1
    for i in range(2, n):
```

```
        if n % i == 0:
            x = 0; break
    return x

# Factorial
def fact(n):
    prod = 1
    while n >= 1:
        prod *= n
        n -= 1
    return prod

# Linear search in array
for i in range(len(x)):
    if s == x[i]:
        print('Found at Position=', i+1)

# Check if number is in range
x = int(input('Enter a number: '))
if x >= 1 and x <= 10:
    print("Between 1 and 10")

# Accept multiple values in one line
a, b = [int(x) for x in input("Enter two numbers: ").split(',')]

# Create 2D matrix from lists
mat = [[1,2,3], [4,5,6], [7,8,9]]
for i in range(len(mat)):
    for j in range(len(mat[i])):
        print(mat[i][j], end=' ')
    print()
---
```

End of Notes -- Units 1 to 4 covered as per syllabus